

Sliding Window — Fundamentals and the Ten Patterns

What this is. A complete study guide for **Sliding Window**, built from a guided reasoning-first session (fundamentals, ten patterns from fixed-size windows through the monotonic deque, and when the technique fails) and **extended** with implementations, harder problems for every pattern, and a full recognition system.

The one idea to keep. Don't ask "*which sliding-window template is this?*" Ask four questions, in order: **What is the window? What state describes it? What makes it invalid? How do I repair it (and when do I update the answer)?** The code should fall out of the answers, not the other way round.

What the original session covered — and what this guide adds

Topic	Original session	This guide
What a window is, left/right pointers, contiguous vs subsequence	✓	✓ (+ diagrams)
Fixed-size vs variable-size, why $O(n)$ despite a for+while	✓	✓ (+ a from-scratch $O(n^2)$ counter-example)
Pattern 1 — Fixed-size window (max sum of k elements)	✓	✓ (+ Maximum Average Subarray, distinct-elements variant)
Pattern 2 — Longest valid window (no repeats)	✓	✓ full solution, traced
Pattern 3 — Longest + frequency map (at most K distinct)	✓	✓ (+ Fruit Into Baskets as the same pattern in disguise)
Pattern 4 — Shortest valid window, and why positive numbers matter	✓	✓ (+ Minimum Window Substring, Permutation in String)
Pattern 5 — Counting valid windows (<code>count += right - left + 1</code>)	✓	✓
Pattern 6 — "At most K" → "exactly K"	✓	✓ (+ Subarrays With K Different Integers)
Pattern 7 — Monotonic deque (Sliding Window Maximum)	✓	✓ (+ Sliding Window Minimum, the mirror pattern)
Pattern 8 — Multiple conditions in one window	✓ (concept)	✓ full worked example + Longest Subarray With Absolute Diff $\leq$ Limit
Pattern 9 — When sliding window fails (negatives, non-contiguous, no incremental state, no safe shrink)	✓	✓ (+ a working fix via prefix sums + monotonic deque, and a from-scratch $O(n^2)$ illustration)
Hard problems per pattern (the session's own consolidation pass)	✓	✓ all solved and tested
Decision procedure, recognition table, complexity notes	✓	✓

All 20+ solutions in this guide are tested against a brute-force reference on hundreds of random inputs, and every step table is produced by actually running the code.

A clarification worth stating up front. The session's Longest Repeating Character Replacement derivation notes that `max_freq` is *allowed to be stale* (it's never decreased when the window shrinks) and asserts this is safe without fully justifying why. §3.3 spells out the argument: a stale `max_freq` can only make the window look *more* restrictive than it really is, which can shrink the window further than strictly necessary but can never record a *longer* window than is actually valid — so the maximum length found is still correct.

Part 1 · Fundamentals

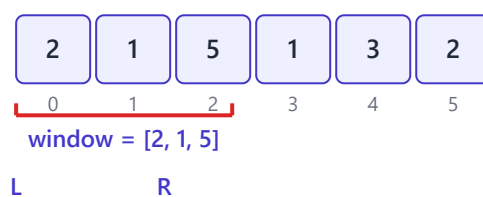
1.1 What is a window?

A window is a **contiguous** portion of an array or string, represented by two pointers:

```
nums = [2, 1, 5, 1, 3, 2]
      L   R
```

Everything between `left` and `right` (inclusive) belongs to the window: `[left ... right]`.

A window is a contiguous range `[left ... right]` inside the array



`left` controls the START of the window; `right` controls the END

1.2 What the two pointers mean

- `right` **expands** the window — bringing a new element into consideration.
- `left` **shrinks** the window — removing the oldest element from consideration.

`right` EXPANDS the window; `left` SHRINKS it



both pointers normally only move FORWARD — that's the source of $O(n)$

Both pointers normally only move forward. Neither one ever goes backward. That single fact is the reason the technique can run in $O(n)$ — see §1.5.

1.3 Fixed-size vs variable-size

	Fixed-size	Variable-size
Size rule	The problem states it: exactly <code>K</code> elements	No fixed size — a condition decides how big the window may grow
Example	"Maximum sum of a subarray of size <code>K</code> "	"Longest subarray with sum ≤ 7 "

Fixed size: `K` is given. Variable size: the **CONDITION** decides the size

fixed (`K = 3`):



variable ($\text{sum} \leq 7$):



1.4 The most important idea

Sliding window works when we can maintain information about a contiguous range while its boundaries move through the input.

The word **contiguous** is doing all the work in that sentence:

```
[1, 5, 1]   contiguous   ✓ candidate for sliding window
[2, 5, 3]   NOT contiguous (a subsequence-style selection) ✗
```

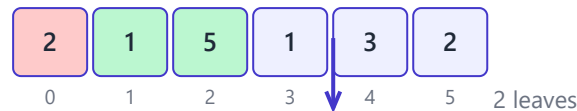
"Is the answer contiguous?" should be the very first question you ask — but it is necessary, not sufficient. We'll see in Part 9 that some contiguous problems still can't use sliding window.

1.5 Why it's O(n): reuse, don't recompute

Don't recompute the entire window. Update the previous window's state.

For `nums = [2, 1, 5, 1, 3, 2]`, `k = 3`, moving from `[2, 1, 5]` to `[1, 5, 1]`:

Don't recompute the window — reuse the previous state



sum = 8



new_sum = 8 - 2 + 1 = 7

```
new_sum = old_sum - (element leaving) + (element entering)
          = 8 - 2 + 1
          = 7
```

Seeing `for right in range(n): while ...: left += 1` should **not** trigger an automatic " $O(n^2)$!" reaction:

Why the `for + while` is still $O(n)$: each pointer crosses the array

right: moves forward at most n times



left: moves forward at most n times



total pointer movements $\leq n + n = 2n = O(n)$

the KEY property: every element enters the window once, and leaves it at r

`right` moves forward from the start to the end **at most once**; `left` also moves forward **at most once**. Total movements $\leq 2n \rightarrow O(n)$. The real principle underneath the arithmetic: **each element enters the window at most once and leaves it at most once**. This argument reappears for every pattern in this guide, including the counting patterns (Part 5) and the monotonic deque (Part 7).

1.6 The five questions to ask for every problem

Don't memorise a template. Ask:

1. What does this window represent?
2. What happens when I move `right`?
3. What information changed?
4. When do I need to move `left`?
5. What must remain true about the window? (the invariant)

We will answer these five questions for every pattern in this guide, starting with the fixed-size window in Part 2.

Part 2 · Pattern 1 — Fixed-Size Window

Recognition. *The problem asks about a contiguous range of exactly k elements.*

2.1 Maximum Sum Subarray of Size k

```
nums = [2, 1, 5, 1, 3, 2], k = 3 → answer 9 (from [5, 1, 3]).
```

The five questions.

1. Window: exactly k contiguous elements, `nums[right-k+1 ... right]`.
2. State: the running sum.
3. Moving `right`: add the new element, then check whether the window has grown past size k .
4. Moving `left`: once the window exceeds size k , remove the leftmost element — removing anything else would break contiguity.
5. Invariant: *whenever we evaluate the window, it contains exactly k elements.*

right	window	sum	best so far
2	[2, 1, 5]	8	8
3	[1, 5, 1]	7	8
4	[5, 1, 3]	9	9
5	[1, 3, 2]	6	9

```
def max_sum_subarray(nums, k):
    """The introductory problem. Reuse the previous window's sum instead of recomputing it:
    new_sum = old_sum - leaving + entering."""
    window_sum = 0
    best = float("-inf")
    for right in range(len(nums)):
        window_sum += nums[right]
        if right >= k - 1:
            best = max(best, window_sum)
            window_sum -= nums[right - k + 1]
    return best
```

Complexity. $O(n)$ time, $O(1)$ space — the fixed-window skeleton that every later pattern with a size constraint builds on.

2.2 Maximum Average Subarray I (LeetCode 643)

The same fixed window; divide the best sum by k at the end.

```
def max_average_subarray(nums, k):
    """LeetCode 643. Same fixed-window sum, just divided by k at the end."""
    return max_sum_subarray(nums, k) / k
```

2.3 Harder: Maximum Sum of a Distinct Subarray of Size K

`nums = [1, 5, 4, 2, 9, 9, 9]`, `k = 3` → windows `[4, 2, 9] = 15` is the best **valid** one (`[2, 9, 9]` and `[9, 9, 9]` both repeat a value and are excluded).

Still a **fixed** window — every candidate has exactly `k` elements — but now with an extra validity condition: **all `k` must be distinct**. So the state grows from one number (the sum) to two: the sum, and a frequency map that tells us whether the window is currently all-distinct.

```
def maximum_distinct_subarray_sum(nums, k):
    """Maximum Sum of a Distinct Subarray of Size K. Still a fixed window (exactly k elements),
    with an extra validity condition (all k must be distinct) carried in a frequency map."""
    left = 0
    current_sum = 0
    count = {}
    best = 0
    for right in range(len(nums)):
        x = nums[right]
        current_sum += x
        count[x] = count.get(x, 0) + 1
        if right - left + 1 > k:                                # keep the window's SIZE at k
            y = nums[left]
            current_sum -= y
            count[y] -= 1
            if count[y] == 0:
                del count[y]
            left += 1
        if right - left + 1 == k and len(count) == k:          # exactly k elements AND all distinct
            best = max(best, current_sum)
    return best
```

What makes it harder than the basic version. The core recognition — *exactly K elements* → *fixed window* — doesn't change. What changes is that a fixed window can still have its own validity condition layered on top, the same way a variable window does.

Part 3 · Pattern 2 — Longest Valid Window

Recognition. Find the longest contiguous range satisfying a condition.

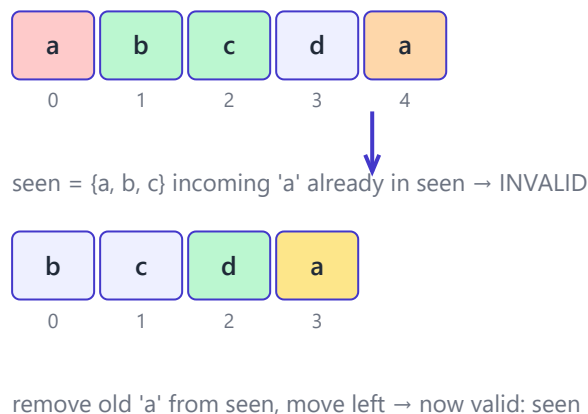
3.1 Longest Substring Without Repeating Characters (LeetCode 3)

```
s = "abcabcbb" → answer 3 ( "abc" ).
```

The five questions.

1. Window: a contiguous substring `s[left:right+1]`.
2. State: **which characters are currently inside the window** — a set is enough here, because no character may repeat at all.
3. Invalid: the incoming character already exists in the window.
4. Repair: remove from the left **while** the duplicate is still present — not `if`, because one removal may not be enough (imagine `[a, b, a, c]` with another `a` arriving: removing the first `a` still leaves a duplicate).
5. Update: after the window is valid, `best = max(best, right - left + 1)`.

Longest Substring Without Repeating Characters: repair on a duplicate



The `+1` in `right - left + 1` is because both endpoints are included: indices `2..4` contain 3 elements (`4 - 2 + 1`).

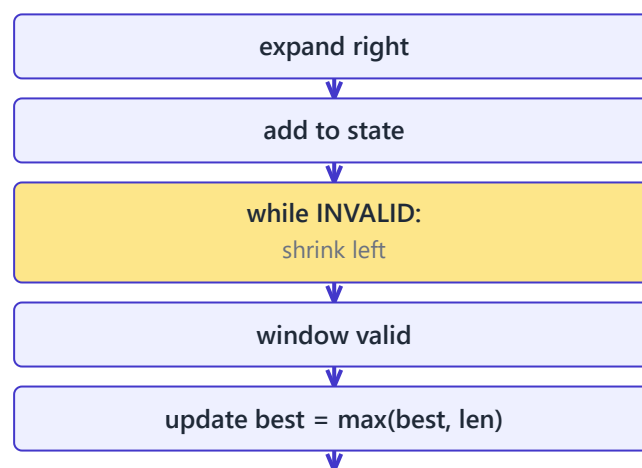
s[right]	removed	window	length	best
a	-	a	1	1
b	-	ab	2	2
c	-	abc	3	3
a	a	bca	3	3
b	b	cab	3	3
c	c	abc	3	3
b	a, b	cb	2	3
b	c, b	b	1	3

```
def longest_unique_substring(s):
    """LeetCode 3. State = the set of characters currently in the window. Invalid = the
    incoming character is already in that set. Shrink from the left until it isn't."""
    left = 0
    seen = set()
    best = 0
    for right in range(len(s)):
        while s[right] in seen:
            seen.remove(s[left])
            left += 1
        seen.add(s[right])
        best = max(best, right - left + 1)
    return best
```

Complexity. $O(n)$ time (\$1.5's argument again), $O(\min(n, \text{alphabet size}))$ space.

The pattern, stripped of this specific problem: expand $\rightarrow$ if invalid, shrink until valid $\rightarrow$ update the maximum. This is the **Longest Valid Window** skeleton, reused for every problem in this Part.

Longest valid window: shrink WHILE INVALID, then update the



3.2 Longest Substring with At Most K Distinct Characters

`s = "eceba"` , `k = 2` $\rightarrow$ answer 3 ("ece").

What's different from §3.1? Repetition is now completely allowed — "eeeeee" has only 1 distinct character and is valid for any `k  $\geq$  1`. The question isn't "does this exist?" but "how many different characters, and could removing one still leave others behind?"

A SET tells you presence; a frequency MAP tells you how many are left

no repeats allowed → a set is enough

{a, b, c}

at most K distinct → repeats are fine, need counts

{e: 2, c: 1}

removing one 'e' still leaves count 1 — 'e' stays in the window

So the state upgrades from a **set** to a **frequency map**: `count = {'e': 2, 'c': 1}`. Number of distinct characters = `len(count)`.

The subtlety in shrinking. When `left` moves past a character, decrement its count — but only delete the key once the count reaches 0:

```
count[s[left]] -= 1
if count[s[left]] == 0:
    del count[s[left]]
```

This keeps `len(count)` an accurate count of *distinct* characters, not just characters ever seen.

s[right]	count after	shrunk	window	best
e	{'e': 1}	-	e	1
c	{'e': 1, 'c': 1}	-	ec	2
e	{'e': 2, 'c': 1}	-	ece	3
b	{'e': 1, 'b': 1}	e, c	eb	3
a	{'b': 1, 'a': 1}	e	ba	3

```
def longest_substring_k_distinct(s, k):
    """Longest substring with at most K distinct characters. A SET is no longer enough because
    repetition is allowed; we need a frequency MAP so removing one occurrence doesn't drop a
    character that still has others left in the window."""
    left = 0
    count = {}
    best = 0
    for right in range(len(s)):
        count[s[right]] = count.get(s[right], 0) + 1
        while len(count) > k:                                # invalid: too many DISTINCT characters
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best
```

The general lesson. *The sliding-window mechanism doesn't change. The state you maintain changes according to the constraint.* A set answers "does it exist"; a frequency map answers "how many, and does one still remain after removal."

Fruit Into Baskets (LeetCode 904) is the same problem with different words: two baskets, one fruit type each, picking a contiguous run of trees is exactly *at most 2 distinct fruit types*.

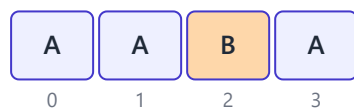
```
def total_fruit(fruits):  
    """LeetCode 904 ('Fruit Into Baskets'): exactly the at-most-2-distinct problem in disguise -  
    two baskets = at most 2 distinct fruit types in a contiguous run of trees."""  
    return longest_substring_k_distinct(fruits, 2)
```

3.3 Harder: Longest Repeating Character Replacement (LeetCode 424)

You may replace **at most** k characters. Find the longest substring that can become all-one-character. $s =$ "AABABBA", $k = 1 \rightarrow$ answer 4 ("AABA" $\rightarrow$ "AAAA").

Deriving the validity condition. If the window has length 7 and its most frequent character occurs 4 times, turning the rest into that character costs $7 - 4 = 3$ replacements:

Longest Repeating Character Replacement: replacements needed



window length = 4, most frequent (A) = 3

replacements needed = length - max_freq = 4 - 3 = 1

valid when window_length - max_freq $\leq k$

```
replacements_needed = window_length - max_frequency  
valid when:          window_length - max_frequency <= k
```

s[right]	max_freq (may be stale)	replacements needed	window	best
A	1	0	A	1
A	2	0	AA	2
B	2	1	AAB	3
A	3	1	AABA	4
B	3	1	ABAB	4
B	3	1	BABB	4
A	3	1	ABBA	4

```
def character_replacement(s, k):
    """LeetCode 424. To turn the whole window into one repeated character we must replace
    window_size - max_frequency characters. Valid when that is <= k.
    max_freq is allowed to be a stale (historical) high - it can only ever make the window look
    MORE restrictive than it truly is, never less, so the recorded best length is still correct."""
    left = 0
    count = {}
    max_freq = 0
    best = 0
    for right in range(len(s)):
        ch = s[right]
        count[ch] = count.get(ch, 0) + 1
        max_freq = max(max_freq, count[ch])
        while (right - left + 1) - max_freq > k:           # replacements needed > k
            count[s[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best
```

The subtlety. `max_freq` is **never decreased** when the window shrinks — it can lag behind the window's *true* current maximum frequency. Why is this safe?

If `max_freq` overstates the true maximum, the validity check `window_length - max_freq <= k` is **stricter** than it needs to be — so the algorithm might shrink a window that was technically still valid. It can never do the opposite (accept a window that is actually invalid), because a real, larger frequency was genuinely seen at that window length at some earlier point, and `best` is only ever updated with `right - left + 1` for windows that passed the (possibly too-strict) check. The maximum length recorded is therefore always achievable, even though some intermediate windows are shrunk slightly more eagerly than required.

What makes it hard. The validity condition itself has to be *derived* — it isn't handed to you the way "sum ≤ target" is.

3.4 Max Consecutive Ones III (LeetCode 1004)

Flip at most `k` zeros to ones; find the longest run of ones. The same longest-window skeleton, with `zeros` in the window as the state.

```
def max_consecutive_ones_iii(nums, k):  
    """LeetCode 1004: longest subarray of 1s after flipping at most k zeros. State = zeros seen;  
    invalid when zeros > k. The same "longest valid window" skeleton as the two problems above."""  
    left = 0  
    zeros = 0  
    best = 0  
    for right in range(len(nums)):  
        if nums[right] == 0:  
            zeros += 1  
        while zeros > k:  
            if nums[left] == 0:  
                zeros -= 1  
            left += 1  
        best = max(best, right - left + 1)  
    return best
```

Part 4 · Pattern 3 — Shortest Valid Window

Recognition. Find the shortest contiguous range satisfying a condition — the mirror image of Part 3.

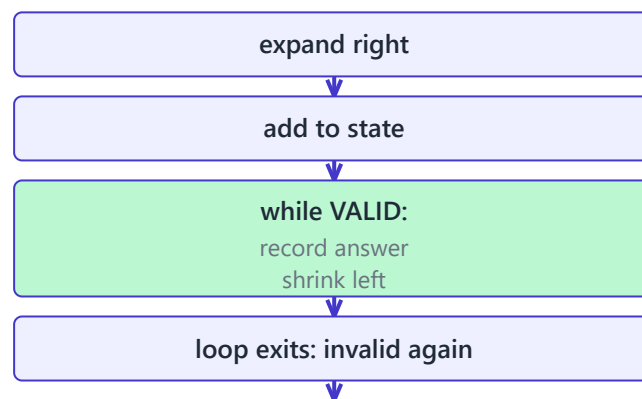
4.1 Minimum Size Subarray Sum (LeetCode 209)

Positive integers, target = 7. nums = [2, 3, 1, 2, 4, 3] → answer 2 ([4, 3]).

The five questions.

1. Window: `nums[left:right+1]`.
2. State: the running sum.
3. Valid: `sum >= target`.
4. **Repair the other way round:** once valid, we want to shrink **while still valid**, recording the length **before** each shrink — because we want it as small as possible, not merely valid.
5. Update: every time the window is valid (inside the `while`), not just once after the loop.

Shortest valid window: shrink WHILE VALID, recording before



The mental model that makes this obvious. "Once this window becomes valid, I want to make it as small as possible." That single sentence produces `while valid: record; shrink` — the reverse of Part 3's `while invalid: shrink; then record`.

	Longest valid	Shortest valid
Shrink while...	invalid	valid
Record the answer...	after the loop (window just became valid)	inside the loop, before each shrink

right	window	sum	length recorded
3	[2, 3, 1, 2]	8	4
4	[3, 1, 2, 4]	10	4
4	[1, 2, 4]	7	3
5	[2, 4, 3]	9	3

right	window	sum	length recorded
5	[4, 3]	7	2

```
def min_subarray_len(target, nums):
    """LeetCode 209. Positive integers only (see Part 9 for why). Shrink WHILE valid (the reverse
    of the longest-window skeleton), recording the length BEFORE each shrink."""
    left = 0
    current_sum = 0
    best = float("inf")
    for right in range(len(nums)):
        current_sum += nums[right]
        while current_sum >= target:
            best = min(best, right - left + 1)
            current_sum -= nums[left]
            left += 1
    return 0 if best == float("inf") else best
```

4.2 Why positive numbers matter here

With **positive** values, expanding the window can only increase the sum, and shrinking it can only decrease the sum — a clean, predictable, **monotonic** relationship. That monotonicity is exactly what licenses "shrink while valid, and stop the instant it's invalid — nothing further left could help."

Negative values destroy this. `nums = [4, -3, 6]`: the full window sums to 7 (valid), but shrinking to `[-3, 6]` gives 3 (invalid) even though the array clearly contains other valid, possibly shorter windows elsewhere. Part 9 works through exactly why this breaks the algorithm's *guarantee*, not just its intuition, and shows the fix.

The principle to take forward: *"It's contiguous" does not automatically mean "use sliding window."* Sliding window additionally needs a **monotonic relationship** between the boundaries moving and the validity condition.

4.3 Minimum Window Substring (LeetCode 76) — Hard

Smallest substring of `s` containing every character of `t`, with duplicates. `s = "ADOBECODEBANC"`, `t = "ABC"` → answer `"BANC"`.

Deriving the state. `need = Counter(t)` (e.g. `A: 2` if `t` had two A's). As characters enter the window we track `have`. Comparing two whole maps on every step would be wasteful, so instead maintain **formed** — how many of the *distinct* required characters currently have **enough** copies in the window:

```
ch in need and have[ch] == need[ch] → formed += 1 (this requirement just became satisfied)
```

The window is valid exactly when `formed == required` (`required = len(need)`, the number of distinct characters `t` needs).

right	valid window	formed	note
10	ECODEBA	3	-
10	CODEBA	3	record
12	ODEBANC	3	-
12	DEBANC	3	record
12	EBANC	3	record
12	BANC	3	record

Shortest window containing all of ABC : **BANC**

```
def min_window(s, t):
    """LeetCode 76. 'need' = required counts from t; 'have' = counts in the window; 'formed'
    counts how many DISTINCT required characters are currently fully satisfied (not scanning
    the whole map every time). Valid when formed == required; then shrink aggressively."""
    if not s or not t:
        return ""
    need = Counter(t)
    have = {}
    required = len(need)
    formed = 0
    left = 0
    best_len = float("inf")
    best_start = 0
    for right in range(len(s)):
        ch = s[right]
        have[ch] = have.get(ch, 0) + 1
        if ch in need and have[ch] == need[ch]:
            formed += 1
        while formed == required:
            window_len = right - left + 1
            if window_len < best_len:
                best_len = window_len
                best_start = left
            left_ch = s[left]
            have[left_ch] -= 1
            if left_ch in need and have[left_ch] < need[left_ch]:
                formed -= 1
            left += 1
    return "" if best_len == float("inf") else s[best_start:best_start + best_len]
```

Complexity. $O(|s| + |t|)$ time, $O(|s| + |t|)$ space.

What makes it hard. The basic shortest-window problem needed one number (a sum). This needs a *frequency requirement* (`need`), a *running count* (`have`), and a *derived satisfaction counter* (`formed`) — three pieces of state instead of one — but the skeleton underneath (`expand` → `become valid` → `record` → `shrink while valid`) is identical to §4.1.

4.4 Permutation in String (LeetCode 567)

Does `s2` contain a permutation of `s1` (a rearrangement, same multiset of characters) anywhere as a contiguous substring?

A permutation of `s1` is exactly a substring with the **same character frequencies**, which is Part 5's problem (Find All Anagrams) with an early exit:

```
def permutation_in_string(s1, s2):  
    """LeetCode 567: does s2 contain a permutation of s1? A permutation of s1 is just an anagram  
    with the same multiset of characters, so this is find_all_anagrams (Part 4) with an early  
    exit."""  
    return len(find_all_anagrams(s2, s1)) > 0
```

Part 5 · Pattern 4 — Frequency-Map Window (Fixed Size)

Recognition. A *fixed-size window* whose *entire frequency distribution must match a target* — not just "contains" or "sum".

5.1 Find All Anagrams in a String (LeetCode 438)

```
s = "cbaebabacd", p = "abc" → answer [0, 6].
```

An anagram of `p` has (1) the **same length** as `p`, and (2) the **same character frequencies**. So this is a fixed-size window (`len(p)`) plus a frequency-map validity condition — a close cousin of §2.3's distinct-elements fixed window, except the condition here is "matches exactly" rather than "all different."

Avoiding a full map comparison every step. Maintain `matched` — the count of characters whose frequency in the window currently **equals** the required frequency — and update it incrementally as each character's count crosses into or out of agreement:

```
if window[ch] == need[ch]:    matched += 1    # just became satisfied
elif window[ch] == need[ch] + 1: matched -= 1    # was satisfied, now over-shot
```

```

def find_all_anagrams(s, p):
    """LeetCode 438. Fixed-size window (== len(p)) whose FREQUENCY DISTRIBUTION must match `p`'s.
    `matched` tracks how many characters currently have the exact required count, so we never
    compare two whole maps against each other on every step."""
    if len(p) > len(s):
        return []
    need = Counter(p)
    window = {}
    required = len(need)
    matched = 0
    left = 0
    result = []
    for right in range(len(s)):
        ch = s[right]
        window[ch] = window.get(ch, 0) + 1
        if ch in need:
            if window[ch] == need[ch]:
                matched += 1
            elif window[ch] == need[ch] + 1:
                matched -= 1
        if right - left + 1 > len(p):  # keep the window's size == len(p)
            left_ch = s[left]
            if left_ch in need:
                if window[left_ch] == need[left_ch]:
                    matched -= 1
                elif window[left_ch] == need[left_ch] + 1:
                    matched += 1
            window[left_ch] -= 1
            if window[left_ch] == 0:
                del window[left_ch]
            left += 1
        if right - left + 1 == len(p) and matched == required:
            result.append(left)
    return result

```

Complexity. $O(n)$ time, $O(\text{alphabet size})$ space.

What makes frequency-map windows their own pattern. The question shifts from "is this character present?" (a set) to "does the entire distribution match?" (a map plus a running satisfaction counter) — the same upgrade path as §3.2, but for a fixed-size window instead of a variable one.

Part 6 · Pattern 5 — Counting Valid Windows

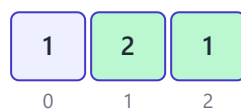
Recognition. Not "the longest" or "the shortest" — "how many contiguous windows satisfy the condition?"

6.1 Count Subarrays With Sum At Most K

Positive integers. `nums = [1, 2, 1]`, `K = 3` → answer 5: `[1]`, `[1,2]`, `[2]`, `[2,1]`, `[1]`.

The key realisation. Once a window `[left...right]` is valid, **every suffix of it** (still ending at `right`) is also valid — because with positive numbers, removing elements from the left can only *decrease* the sum:

Once a window is valid, EVERY suffix ending at `right` is also valid



window `[2, 1]` (indices `1..2`) has sum $3 \leq K=3$ → valid

its suffixes ending at `right=2`: `[2,1]` and `[1]` → both valid, `count += 2`

`count += (right - left + 1)` adds all of them at once, no double count (unique

window `[2, 1]` (sum 3, valid) ⇒ its suffixes `[2,1]` and `[1]` are BOTH valid

So instead of enumerating those suffixes one at a time, add all of them at once: `count += right - left + 1`.

Why nothing gets double-counted. Every subarray has a **unique** ending index. `right - left + 1` at a given `right` counts *only* the valid subarrays that end **exactly** at that `right` — a different `right` can never re-count the same subarray, even though the underlying elements overlap.

right	valid window	sum	suffixes added	running count
0	<code>[1]</code>	1	1	1
1	<code>[1, 2]</code>	3	2	3
2	<code>[2, 1]</code>	3	2	5

```
def count_subarrays_at_most_k(nums, k):  
    """Count subarrays with sum <= k (positive integers). Once a window is valid, EVERY suffix of  
    it (ending at the same `right`) is also valid, so we add right-left+1 all at once instead of  
    enumerating them – and no subarray is double-counted because each has a unique right  
    endpoint."""  
    left = 0  
    current_sum = 0  
    count = 0  
    for right in range(len(nums)):  
        current_sum += nums[right]  
        while current_sum > k:  
            current_sum -= nums[left]  
            left += 1  
        count += right - left + 1  
    return count
```

This relies on the same positive-number monotonicity as Part 4's shortest-window pattern: with negatives, a shrunk window is no longer guaranteed valid, so "*every suffix of a valid window is valid*" would no longer hold.

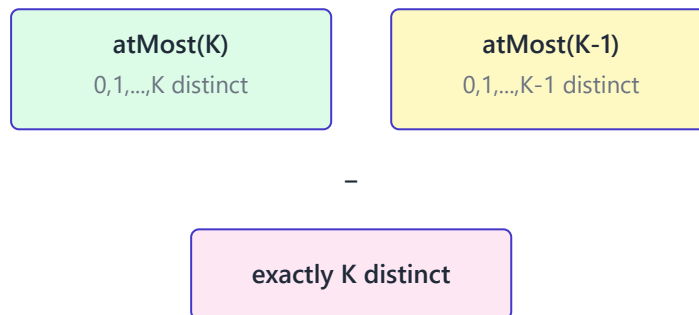
Part 7 · Pattern 6 — "At Most K" → "Exactly K"

Recognition. Counting subarrays with exactly K of something is awkward directly — but "at most K " is the easy, monotonic version we already have from Part 6.

7.1 The transformation

$$\text{exactly}(K) = \text{atMost}(K) - \text{atMost}(K - 1)$$

$$\text{exactly}(K) = \text{atMost}(K) - \text{atMost}(K-1)$$



the 0..K-1 bucket cancels out,
leaving only subarrays with EXACTLY K distinct values

Why this works. Classify every subarray by how many "somethings" (distinct values, odd numbers, ...) it contains. `atMost(K)` includes everything with `0, 1, ..., K` of them; `atMost(K-1)` includes everything with `0, 1, ..., K-1`. Subtracting cancels every bucket **except exactly K** — because every subarray counted in `atMost(K-1)` is also counted in `atMost(K)`, and nothing with more than `K` appears in either.

The mental model. *Exactly K = everything with $\leq K$, minus everything with $\leq K-1$.* Not a formula to memorise blindly — a consequence of how the buckets nest.

7.2 Count Number of Nice Subarrays (LeetCode 1248)

Count subarrays with **exactly k odd numbers**. `nums = [1, 1, 2, 1, 1]`, `k = 3` → answer 2.

`atMost(limit)` is the Part 6 counting pattern with `odd_count` as the state:

quantity	value
<code>atMost(3)</code>	14
<code>atMost(2)</code>	12
<code>exactly(3) = atMost(3) - atMost(2)</code>	2

```

def number_of_nice_subarrays(nums, k):
    """LeetCode 1248. 'Exactly k odd numbers' is awkward directly; exactly(k) = atMost(k) -
    atMost(k-1)
    turns it into two easy monotonic counting problems."""
    def at_most(limit):
        if limit < 0:
            return 0
        left = 0
        odd_count = 0
        result = 0
        for right in range(len(nums)):
            if nums[right] % 2 == 1:
                odd_count += 1
            while odd_count > limit:
                if nums[left] % 2 == 1:
                    odd_count -= 1
                left += 1
            result += right - left + 1
        return result

    return at_most(k) - at_most(k - 1)

```

7.3 Harder: Subarrays With K Different Integers (LeetCode 992)

Count subarrays with **exactly k distinct** integers. `nums = [1, 2, 1, 2, 3]`, `k = 2` → answer `7`.

Same transformation, `atMost` implemented with a frequency map (Part 6's counting pattern, upgraded the same way §3.2 upgraded a set to a map):

```

def subarrays_with_k_distinct(nums, k):
    """LeetCode 992. Same atMost(k) - atMost(k-1) transformation, with a frequency map this time."""
    def at_most(limit):
        if limit < 0:
            return 0
        left = 0
        freq = {}
        result = 0
        for right in range(len(nums)):
            x = nums[right]
            freq[x] = freq.get(x, 0) + 1
            while len(freq) > limit:
                y = nums[left]
                freq[y] -= 1
                if freq[y] == 0:
                    del freq[y]
                left += 1
            result += right - left + 1
        return result

    return at_most(k) - at_most(k - 1)

```

The deeper lesson. You should now recognise this shape immediately: "exactly K " $\rightarrow$ ask "can I get this as $\text{atMost}(K) - \text{atMost}(K-1)$?" If yes, an awkward exact condition becomes two ordinary, monotonic sliding-window counting problems.

Part 8 · Pattern 7 — Monotonic Deque

Recognition. Fixed-size window + need the maximum (or minimum) of every window, efficiently.

8.1 Why the ordinary window state doesn't work

For a sum, `new_sum = old_sum - leaving + entering` works because subtraction is a cheap, exact undo. But for a **maximum**: if the maximum element leaves the window, what's the new maximum? There's no cheap way to "undo" a max — you'd need to rescan.

8.2 Sliding Window Maximum (LeetCode 239)

```
nums = [1, 3, -1, -3, 5, 3, 6, 7], k = 3 → answer [3, 3, 5, 5, 6, 7].
```

The idea: dominance. Suppose the deque (by value) is `[5, 3]` and a new value `6` arrives. Since `6 > 3`, and `6` is **newer than** `3` (arrived later, so it will still be in the window after `3` has left), `3` can **never** be the maximum of any future window that also contains `6`. It is permanently useless and can be discarded.

Dominance: a bigger, newer value makes smaller older ones useless



deque (values): `[5, 3]`

new value 6 arrives: `6 > 3` → pop 3. `6 > 5`? no → stop. deque becomes `[5, 6]`

3 can never be the max again: 6 is bigger AND will stay in the window longer

Two removal rules:

Two removal rules for the monotonic deque



invariant: the deque holds only useful, non-expired candidates, in decreasing value

→ **the FRONT is always the maximum of the current window**

- Back — dominated:** `while dq and nums[dq[-1]] <= nums[right]: dq.pop()` — a value that is smaller (or equal) and older than the newcomer can never win again.
- Front — expired:** `if dq[0] <= right - k: dq.popleft()` — its index has fallen outside the current window.

Why store indices, not values? Because rule 2 needs to know *where* a candidate is, to tell whether it has aged out of the window. The values themselves are recovered with `nums[dq[i]]`.

Invariant. The deque holds only useful, non-expired candidates, indices in the window, values strictly decreasing front-to-back. Therefore `nums[dq[0]]` is always the current window's maximum.

nums[right]	popped (dominated, back)	popped (expired, front)	deque values (front→back)	window max
1	-	-	[1]	-
3	1	-	[3]	-
-1	-	-	[3, -1]	3
-3	-	-	[3, -1, -3]	3
5	-3, -1, 3	-	[5]	5
3	-	-	[5, 3]	5
6	3, 5	-	[6]	6
7	6	-	[7]	7

```
def max_sliding_window(nums, k):
    """LeetCode 239. The deque holds INDICES with strictly decreasing values; a new value that
    dominates the back (>=) makes every dominated index permanently useless, since the new one
    is both bigger AND stays in the window longer. Front = current window maximum."""
    dq = deque()
    result = []
    for right in range(len(nums)):
        while dq and nums[dq[-1]] <= nums[right]:          # dominated: strictly smaller AND older
            dq.pop()
        dq.append(right)
        if dq[0] <= right - k:                             # expired: outside the window
            dq.popleft()
        if right >= k - 1:
            result.append(nums[dq[0]])
    return result
```

Complexity. Each index enters the deque exactly once and leaves it **at most once** (from either end) — the same "enters once, leaves once" argument from §1.5 — giving **O(n)** time despite the nested `while` loops, and **O(k)** space.

8.3 Sliding Window Minimum

The mirror image: flip both comparisons and keep an **increasing** deque.

```
def min_sliding_window(nums, k):  
    """The mirror image: an increasing deque gives the window MINIMUM."""  
    dq = deque()  
    result = []  
    for right in range(len(nums)):  
        while dq and nums[dq[-1]] >= nums[right]:  
            dq.pop()  
        dq.append(right)  
        if dq[0] <= right - k:  
            dq.popleft()  
        if right >= k - 1:  
            result.append(nums[dq[0]])  
    return result
```

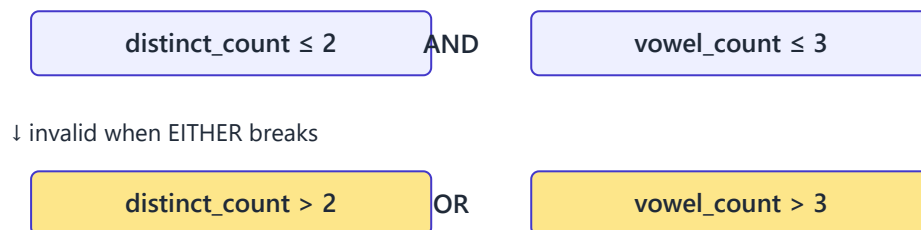
Part 9 · Pattern 8 — Multiple Conditions in One Window

Recognition. *More than one validity rule applies to the same window at once.*

9.1 The AND/OR rule

Suppose a window must satisfy **both**: at most 2 distinct characters, **and** at most 3 vowels.

Multiple conditions: validity is an AND, the shrink trigger is an OR



while `(distinct_count > 2) or (vowel_count > 3)`: shrink

```
VALID    when    distinct_count <= 2    AND    vowel_count <= 3
INVALID  when    distinct_count > 2     OR     vowel_count > 3
```

Validity is an AND of every condition. The shrink trigger is an OR of their negations. Everything else — expand, update state, `while` invalid: shrink, update the answer once valid — is exactly the Part 3 longest-window skeleton; only the condition got richer.

```
def longest_substring_2_distinct_and_vowels(s, max_distinct, max_vowels):
    """Illustrates combining two independent constraints in one window: distinct character count
    AND vowel count. Validity is an AND of both; invalidity (the shrink condition) is an OR."""
    vowels = set("aeiouAEIOU")
    left = 0
    freq = {}
    vowel_count = 0
    best = 0
    for right in range(len(s)):
        ch = s[right]
        freq[ch] = freq.get(ch, 0) + 1
        if ch in vowels:
            vowel_count += 1
        while len(freq) > max_distinct or vowel_count > max_vowels:
            left_ch = s[left]
            freq[left_ch] -= 1
            if freq[left_ch] == 0:
                del freq[left_ch]
            if left_ch in vowels:
                vowel_count -= 1
            left += 1
        best = max(best, right - left + 1)
    return best
```

The mental rule to keep: don't ask *"which template is this?"* — ask *"what information must I maintain so I can tell whether the window satisfies every constraint?"* Then: state describes the window, constraints define validity, a violation of **any** constraint triggers a shrink.

9.2 Harder: Longest Subarray With Absolute Diff $\leq$ Limit (LeetCode 1438)

`nums = [10, 1, 2, 4, 7, 2]`, `limit = 5` → answer 4 (`[2, 4, 7, 2]`, since `7 - 2 = 5`).

The validity condition is `max(window) - min(window) <= limit`. Maintaining a running max **and** min efficiently, as the window both grows and shrinks, needs **two monotonic deques at once** — a decreasing one for the max (Part 8) and an increasing one for the min:

$\text{max} - \text{min} \leq \text{limit}$ needs BOTH a max-deque and a min-deque



window [2,4,7,2] (indices 3..5 shown + one more): max deque front = 7, min deque

$7 - 2 = 5 \leq \text{limit}(5) \rightarrow \text{valid, length 4}$

```
def longest_subarray_limit(nums, limit):
    """LeetCode 1438. Validity needs BOTH the window max and min, updated as the window moves -
    one decreasing deque for the max, one increasing deque for the min."""
    max_dq, min_dq = deque(), deque()
    left = 0
    best = 0
    for right in range(len(nums)):
        while max_dq and nums[max_dq[-1]] <= nums[right]:
            max_dq.pop()
        max_dq.append(right)
        while min_dq and nums[min_dq[-1]] >= nums[right]:
            min_dq.pop()
        min_dq.append(right)
        while nums[max_dq[0]] - nums[min_dq[0]] > limit:
            if max_dq[0] == left:
                max_dq.popleft()
            if min_dq[0] == left:
                min_dq.popleft()
            left += 1
        best = max(best, right - left + 1)
    return best
```

What makes it hard. Three ideas layer on top of each other: a variable window, a max-deque, and a min-deque — each contributing part of the state that the single condition `max - min <= limit` depends on. **This is exactly why sliding window shouldn't be thought of as one template:** identify what state the condition needs, then pick the data structure(s) that maintain it efficiently.

Part 10 · Pattern 9 — When Sliding Window Fails

Recognition that failure is coming is as important as recognising the pattern itself. The central test: **can I move left forward and safely discard whatever left the window, forever?**

10.1 Failure case 1 — Negative numbers

With positive numbers: expand → sum increases; shrink → sum decreases. A clean, predictable, monotonic relationship. **Negative numbers destroy it** — adding an element can *decrease* the sum, and removing one can *increase* it.

Negative numbers break the monotonic shrink assumption



$\text{sum}([1, -2, 5]) = 4$, $\text{target} = 4 \rightarrow \text{VALID}$, length 3

shrink once: $\text{sum}([-2, 5]) = 3 \rightarrow \text{invalid}$ — algorithm STOPS shrinking here

but shrinking again: $\text{sum}([5]) = 5 \geq 4$ — valid, length 1 (the TRUE answer, miss)

A concrete miss. `nums = [1, -2, 5]`, `target = 4`: the window `[1, -2, 5]` is valid (sum 4). Shrinking once gives `[-2, 5]` (sum 3) — invalid, so the ordinary algorithm **stops shrinking** and moves on. But shrinking *again* would reach `[5]` alone (sum $5 \geq 4$), the true shortest answer of length 1. The algorithm never gets there, because it assumed "*once shrinking makes the window invalid, no further shrinking can help*" — true for positive numbers, **false** here, since removing `-2` would have increased the sum right back above target.

right	window	sum	length recorded
2	[1, -2, 5]	4	3

The ordinary (shrink-while-valid) scan never even considers `[5]` alone: once it shrinks `[1, -2, 5]` to `[-2, 5]` (sum 3, invalid) it stops shrinking and moves `right` on. It reports length **3**, but the true shortest window is length **1** (`[5]` by itself, sum $5 \geq 4$) — removing the `-2` would have *increased* the sum back above target, which the algorithm never checks. The prefix-sum + monotonic-deque method (below) finds **1** correctly.

Why "you might eventually find an answer anyway" misses the point. The issue isn't that no efficient algorithm exists — it's that *this specific* left-only-ever-moves-forward algorithm has **no guarantee** of finding the true optimum once the monotonic relationship it depends on is gone.

The fix: prefix sums + a monotonic deque. Define `prefix[i] = sum(nums[0:i])`, so `sum(i..j-1) = prefix[j] - prefix[i]`. We want the smallest `j - i` with `prefix[j] - prefix[i] >= k`. Maintain a deque of candidate start indices with **increasing prefix sums** — the same dominance idea as Part 8, but over prefix values instead of raw ones: a later index with an equal-or-smaller prefix always dominates an earlier one as a future start point.

right	prefix[right]	start(s) used (front)	dominated (back)	deque (indices)
0	0	-	-	[0]

right	prefix[right]	start(s) used (front)	dominated (back)	deque (indices)
1	1	-	-	[0, 1]
2	-1	-	1, 0	[2]
3	4	2	-	[3]

Shortest length ≥ 4 : **1**

```
def shortest_subarray_at_least_k(nums, k):
    """LeetCode 862. Negative numbers break "shrink while valid" (removing an element can INCREASE
    the sum), so the ordinary window is not guaranteed correct here. Fix: prefix sums + a monotonic
    deque of candidate start indices with increasing prefix values."""
    n = len(nums)
    prefix = [0] * (n + 1)
    for i in range(n):
        prefix[i + 1] = prefix[i] + nums[i]
    dq = deque()
    best = float("inf")
    for right in range(n + 1):
        while dq and prefix[right] - prefix[dq[0]] >= k: # this start is as good as it gets: use
            it, drop it
            best = min(best, right - dq.popleft())
        while dq and prefix[dq[-1]] >= prefix[right]: # a later start with an equal/smaller
            prefix dominates
            dq.pop()
        dq.append(right)
    return -1 if best == float("inf") else best
```

Complexity. $O(n)$ time, $O(n)$ space.

10.2 Failure case 2 — The problem isn't contiguous

Sliding window represents `[left ... right]`. If a problem allows **skipping** elements (a subsequence, not a subarray/substring), no contiguous window can represent the set of possible choices.

Ask: *does the problem require a contiguous subarray/substring, or can I pick elements while skipping others?* If the latter, sliding window's very premise doesn't apply — reach for DP, greedy, or a different structure. As an extreme illustration, "pick any two elements (not necessarily adjacent) with the largest sum" isn't a window question at all — it's solved directly, in one pass, with no left/right boundary in sight:

```
def max_subsequence_sum_len2(nums):
    """Failure case 2 illustration: 'pick any two elements (not necessarily adjacent) with the
    largest sum' is a SUBSEQUENCE question, not a contiguous one – no left/right window applies.
    It's solved directly (largest + second largest), not with sliding window."""
    top1 = top2 = float("-inf")
    for x in nums:
        if x > top1:
            top1, top2 = x, top1
        elif x > top2:
            top2 = x
    return top1 + top2
```

10.3 Failure case 3 — No incremental state

Sliding window's efficiency depends on updating state cheaply: `old state` → `remove left's contribution` → `add right's contribution` → `new state`. If evaluating the window from scratch is unavoidable — say, a property that costs $O(\text{window size})$ to recompute every single time — the supposed $O(n)$ algorithm silently degrades toward $O(n^2)$:

```
def longest_wo_repeat_naive_would_be_quadratic(s):
    """Failure case 3 illustration: recomputing 'has a duplicate?' by re-scanning the window on
    every step (instead of maintaining a set incrementally) turns an  $O(n)$  idea into  $O(n^2)$ ."""
    n = len(s)
    best = 0
    for left in range(n):
        seen = set()
        right = left
        while right < n and s[right] not in seen:
            seen.add(s[right])
            right += 1
        best = max(best, right - left)
    return best
```

This computes the *same answer* as `longest_unique_substring` (§3.1), but re-scans a fresh set from every `left`, throwing away the incremental "enters once, leaves once" property that made the original $O(n)$.

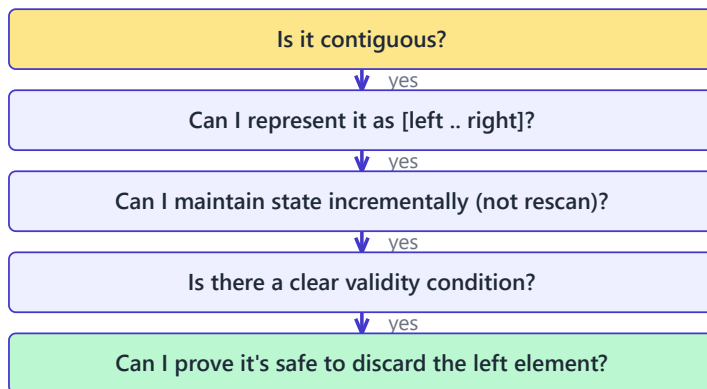
The goal: maintain state incrementally, never recompute the window from scratch.

10.4 Failure case 4 — No safe shrinking rule

The most important question of all: *can I justify that moving `left` forward is guaranteed to move me toward a valid/optimal solution?* If that justification doesn't exist — as in §10.1 — **don't force sliding window onto the problem.**

10.5 The full decision procedure

Should I use sliding window? (each question gates the next)



any "no" along the way:
stop, look for a different technique

→ sliding window is a strong candidate

1. Is the object contiguous?
 2. Can I represent it as [left ... right]?
 3. Can I maintain useful state incrementally (not recompute from scratch)?
 4. Is there a clear validity condition?
 5. When invalid/valid, can I justify moving left forward is safe?
- ↓ all five yes → Sliding Window candidate

The instinct to build: see "*contiguous + moving boundaries + incrementally maintainable state + a safe, justified shrink rule*" and think sliding window — not "*this mentions a substring, so template time.*" If any link in that chain breaks, stop and look for a different technique.

Part 11 · Templates, Complexity and the Hard-Problem Recognition Sheet

11.1 The templates to recognise, not memorise

Fixed-size window

```
window_sum = 0
for right in range(len(nums)):
    window_sum += nums[right]                # expand
    if right >= k - 1:
        evaluate(window_sum)
        window_sum -= nums[right - k + 1]    # shrink: remove the LEFTMOST element
```

Longest valid window

```
left = 0
for right in range(n):
    update_state(add=item[right])
    while invalid():
        update_state(remove=item[left]); left += 1
    best = max(best, right - left + 1)
```

Shortest valid window

```
left = 0
for right in range(n):
    update_state(add=item[right])
    while valid():
        best = min(best, right - left + 1)    # record BEFORE shrinking
        update_state(remove=item[left]); left += 1
```

Counting valid windows

```
left = 0
for right in range(n):
    update_state(add=item[right])
    while invalid():
        update_state(remove=item[left]); left += 1
    count += right - left + 1                # every suffix ending at 'right' is valid too
```

Monotonic deque (maximum)

```

dq = deque()
for right in range(n):
    while dq and nums[dq[-1]] <= nums[right]: # back: dominated
        dq.pop()
    dq.append(right)
    if dq[0] <= right - k: # front: expired
        dq.popleft()
    if right >= k - 1:
        window_max = nums[dq[0]]

```

11.2 The five questions, once more

1. What is the window?
2. What state describes it?
3. What makes it invalid?
4. How do I repair it?
5. When do I update the answer / count — and is the objective *longest*, *shortest*, or a *count*?

11.3 Complexity notes

Pattern	Time	Space
Fixed-size window	$O(n)$	$O(1)$ (or $O(k)$ with a frequency map)
Longest / shortest valid window	$O(n)$ amortized (§1.5)	$O(k)$ or $O(\text{alphabet size})$
Counting valid windows	$O(n)$ amortized	$O(1)$ or $O(k)$
At most $K \rightarrow$ exactly K	$O(n)$ (two $O(n)$ passes)	$O(k)$
Monotonic deque	$O(n)$ amortized (each index in once, out once)	$O(k)$
Prefix sums + deque (negative numbers)	$O(n)$	$O(n)$

11.4 Pitfalls checklist

Pitfall	Fix
<code>if invalid: shrink</code> instead of <code>while</code>	One removal is often not enough — always use <code>while</code> for repair
Deleting a frequency-map key at count 0 forgotten	<code>len(count)</code> silently overcounts distinct elements
Using a set where repetition is allowed	Switch to a frequency map (§3.2)
Assuming "contiguous" alone means sliding window applies	Also need incremental state and a provably safe shrink rule (Part 10)
Shrinking a shortest-window problem only <code>if valid</code> instead of <code>while valid</code>	You'll miss shorter windows nested inside the current valid one

Pitfall	Fix
Recording the shortest-window answer after shrinking, not before	The window right after shrinking may already be invalid
Applying the shortest/counting patterns to arrays with negative numbers	The monotonic relationship breaks — use prefix sums + a monotonic deque instead (§10.1)
Monotonic deque storing values instead of indices	Can't tell when a candidate has expired
Recomputing window properties from scratch each step	Turns an intended $O(n)$ into $O(n^2)$ — maintain state incrementally (§10.3)
Forgetting <code>exactly(K) = atMost(K) - atMost(K-1)</code> needs <code>atMost(-1) = 0</code>	Guard the base case so <code>K = 0</code> works

11.5 The hard-problem recognition sheet

Problem signal	State	Core idea
Exactly K elements	sum / relevant aggregate	Fixed window
Fixed K + uniqueness / distribution match	frequency map	Fixed window + map
Longest satisfying a condition	relevant state	Shrink while invalid
Shortest satisfying a condition	relevant state	Shrink while valid
Must contain all required frequencies	frequency map + satisfaction counter	Shortest valid + map
Count subarrays with condition $\leq K$	relevant state	<code>count += right - left + 1</code>
Count subarrays with condition exactly K	relevant state	<code>atMost(K) - atMost(K-1)</code>
Window maximum / minimum	monotonic deque	Dominance + expiry
Multiple max/min-style constraints at once	multiple deques / combined state	Repair until all conditions hold
Negative numbers + a sum condition	prefix sums (+ deque if needed)	Don't use the ordinary window
Not contiguous (can skip elements)	—	Not sliding window at all

Part 12 · Practice, Recognition Drills and Interview Q&A

12.1 Practice list (LeetCode)

Pattern	Start with	Then	Hard
Fixed-size window	Maximum Average Subarray I (643)	—	Maximum Sum of Distinct Subarray of Size K
Longest valid window	Longest Substring Without Repeating Characters (3)	Max Consecutive Ones III (1004)	Longest Repeating Character Replacement (424)
Frequency map + longest	Fruit Into Baskets (904)	—	Longest Repeating Character Replacement (424)
Shortest valid window	Minimum Size Subarray Sum (209)	Permutation in String (567)	Minimum Window Substring (76)
Frequency map (fixed)	Find All Anagrams in a String (438)	—	—
Counting valid windows	Count subarrays with sum $\leq K$ (custom)	—	Count Number of Nice Subarrays (1248)
At most K $\rightarrow$ exactly K	Count Number of Nice Subarrays (1248)	—	Subarrays With K Different Integers (992)
Monotonic deque	Sliding Window Maximum (239)	Sliding Window Minimum (custom)	—
Multiple conditions	2 distinct + vowel limit (custom)	—	Longest Subarray With Absolute Diff $\leq$ Limit (1438)
When it fails	—	—	Shortest Subarray with Sum at Least K (862)

12.2 Recognition drills

#	Statement	Answer
1	"Longest substring with no repeated characters."	Longest valid window , state = set
2	"Longest substring with at most 2 distinct characters."	Longest valid window , state = frequency map
3	"Smallest subarray with sum $\geq$ target, all positive."	Shortest valid window , shrink while valid
4	"Smallest window containing every letter of another string."	Shortest valid window + frequency requirement (need / have / formed)
5	"How many subarrays have sum at most K?"	Counting : <code>count += right - left + 1</code>
6	"How many subarrays have exactly K odd numbers?"	<code>atMost(K) - atMost(K-1)</code>
7	"Maximum of every window of size K."	Monotonic deque , decreasing
8	"Longest window where max - min $\leq$ limit."	Two monotonic deques (max + min)

#	Statement	Answer
9	"Smallest subarray with sum $\geq K$, array has negatives."	Not the ordinary window — prefix sums + deque
10	"Longest subsequence (not necessarily contiguous) with property X."	Not sliding window — it isn't contiguous

12.3 Interview Q&A

Why is a nested `for` + `while` still $O(n)$ for sliding window? Because `left` and `right` each move forward at most `n` times total across the *entire* run — not per iteration of the outer loop. Every element enters the window once and leaves at most once.

When do you use a set vs. a frequency map as window state? A set when the question is "*does this exist in the window?*" (e.g. no duplicates allowed at all). A frequency map when repetition is allowed but you need to know *how many* — e.g. "at most K distinct" or "matches this exact distribution."

What's the difference in shrink logic between longest-valid and shortest-valid windows? Longest: `while invalid: shrink`, then record once valid. Shortest: `while valid: record`, then `shrink` — you keep shrinking a valid window to see how much smaller it can get, recording each time, until it finally becomes invalid.

Why does counting valid windows use `right - left + 1` instead of checking each subarray? Once `[left..right]` is valid, every suffix of it ending at `right` (i.e. `[left..right]`, `[left+1..right]`, ..., `[right..right]`) is also valid, because shrinking a positive-sum window from the left can only decrease its sum. Each has a unique right endpoint, so summing `right - left + 1` over all `right` counts every valid subarray exactly once.

Explain `exactly(K) = atMost(K) - atMost(K-1)`. `atMost(K)` counts subarrays with 0 through K of the property; `atMost(K-1)` counts 0 through K-1. Subtracting removes every bucket except exactly K.

Why store indices, not values, in a monotonic deque? The values tell you whether a candidate is *useful* (bigger than what's behind it); the index tells you whether it's *still inside the window* (not expired). You need both, and only the index lets you check expiry.

Why can an older, smaller deque candidate be discarded permanently? Because the new value is both **bigger** and **arrived later** — it will out-live the old one in every future window that could contain both, so the old one can never become the maximum again.

Why does sliding window fail with negative numbers? The technique needs expand-increases / shrink-decreases (or the reverse) to hold predictably. With negatives, removing an element from the window can *increase* the sum, so "once invalid while shrinking, stop" is no longer a valid stopping rule — it can permanently skip past the true answer.

Give an example of a contiguous problem where sliding window still isn't the right tool. If maintaining the window's state can't be done incrementally — e.g. it requires re-scanning or re-sorting the whole window on every step — the " $O(n)$ " argument collapses back to $O(n^2)$ or worse, even though the problem is contiguous.

12.4 One-page recap

WINDOW	contiguous range [left ... right]; right expands, left shrinks
WHY $O(n)$	each element enters once, leaves at most once $\rightarrow$ total movement $O(n)$
REUSE STATE	$\text{new_state} = \text{old_state} - \text{leaving} + \text{entering}$ (never recompute from scratch)
FIXED SIZE	window size == K (given)
LONGEST VALID	while INVALID: shrink $\rightarrow$ then record $\max(\text{best}, \text{len})$
SHORTEST VALID	while VALID: record $\min(\text{best}, \text{len})$, THEN shrink
COUNTING	once valid: $\text{count} += (\text{right} - \text{left} + 1)$ -- every suffix ending here is valid too
EXACTLY K	$\text{atMost}(K) - \text{atMost}(K-1)$
MONOTONIC DEQUE	back = dominated (smaller + older, discard); front = expired (outside window)
MULTI-CONDITION	validity = AND of all conditions; shrink trigger = OR of their violations
STATE	set (existence) $\rightarrow$ frequency map (counts) $\rightarrow$ map + satisfaction counter (all-must-match)
WHEN IT FAILS	negative numbers (breaks monotonic sum) $\rightarrow$ prefix sums + deque not contiguous (a subsequence) $\rightarrow$ not sliding window at all no incremental state $\rightarrow$ silently becomes $O(n^2)$ can't prove shrinking left is safe $\rightarrow$ don't force it
ASK	what is the window? what state describes it? what makes it invalid? how do I repair it? longest, shortest, or count? is shrinking left provably safe?